

Resource-Aware Secure Computation Offloading

Yushu Yan Başak Güler

Department of Electrical and Computer Engineering
University of California, Riverside, USA
yyan064@ucr.edu, bguler@ece.ucr.edu

Abstract—Computation offloading enables resource-limited users to delegate resource-intensive processing tasks to more powerful edge devices (workers). In doing so, a major concern is to protect the privacy of sensitive data against untrusted devices. In this work, we propose a secure computation offloading framework to address this challenge. To enable adaptability to resource heterogeneities across the workers, we then propose a hierarchical combinatorial neural multi-armed bandit mechanism for secure computation resource allocation. Our framework provides information-theoretic privacy guarantees for both the sensitive user data and computation results, while minimizing system latency of secure computations in the presence of heterogeneous resource availability. Our experiment results show that the proposed framework achieves $3\times$ faster task completion time compared to conventional secure offloading baselines and is $13\times$ faster than local processing at the user.

Index Terms—Secure computation offloading, coded computing, information-theoretic privacy, neural bandits.

I. INTRODUCTION

Edge computing leverages decentralized compute resources to enable high-performance computing at the network edge. By utilizing powerful edge devices (workers), rather than depending on user devices with limited computational capability, edge computing can significantly reduce data-processing latency [1]–[5]. This is particularly important for applications that require real-time processing and decision-making, such as autonomous vehicles, smart grids, and Internet of Things.

A major challenge in computation offloading is protecting the privacy of sensitive user data, as offloading may allow adversaries to access sensitive user data or computation results. While secure multi-party coded computing mechanisms have been widely studied to address this challenge [6]–[8], the current focus primarily relies on parties with homogeneous resources, while adaptable secure computing frameworks under heterogeneous resources is still underexplored. Recently, resource-aware secure multi-party computation offloading has been explored for linear computation tasks [9] in the context of linear matrix multiplication, by leveraging the well-known Shamir's secret sharing mechanism [10].

Motivated by the recent emergence of complex computational tasks in modern applications such as machine learning, in this work we study resource-aware secure multi-party computation offloading for higher order polynomial computations

(generalizing linear matrix multiplication). We consider the scenario where a resource-limited edge user (data-owner) offloads computations to a group of untrusted workers. In doing so, the user wishes to minimize system latency (task completion time) by adapting the secure offloading mechanism to the heterogeneous communication and compute capabilities of the workers. To that end, we first introduce a scalable privacy-preserving computation offloading mechanism motivated by our recent secure coded computing framework Double Lagrange Coding (DLC), which advances state-of-the-art secure coded computing frameworks by reducing the communication and computation complexity for higher-order polynomial computations [11], [12]. This mechanism enables a resource-limited user to securely offload an encoded version of its dataset to a set of worker devices which are equipped with higher computation capabilities. Each worker then performs computations on the coded dataset (as opposed to the clear dataset), which ensures the information-theoretic privacy of both the data and computation results against any set of up to T adversarial workers, who may collude with one another. The user then collects the coded (local) computation results from the workers to reconstruct the true computation results. DLC enables scalable computation loads for intensive computation tasks, by reducing the minimum number of workers required to perform higher-order polynomial computations.

To adapt the secure computation load of each worker to the heterogeneous communication and computation resources, we then propose a hierarchical combinatorial neural multi-armed bandit (MAB) mechanism. The proposed bandit mechanism is inspired by the recent advances in neural bandits [13], [14], which have demonstrated superior performance to conventional linear contextual bandits in complex decision making tasks. In this work, we propose a hierarchical neural bandit to jointly optimize worker selection and per-worker computation load to minimize end-to-end system latency.

The proposed framework ensures information-theoretic privacy guarantees for both the dataset and computation results. In our experiments, we have tested our framework by implementing a state-of-the-art polynomial neural network architecture (PiNets) [15], and evaluated the performance in terms of both cumulative regret and system latency for a neural network inference task. Extensive experiments demonstrate that the proposed framework significantly reduces the task completion time compared to both conventional multi-party computation offloading baselines and local computation at the user, by up to $3\times$ and $13\times$, respectively.

Research was sponsored in part by the OUSD(R&E)/RT&L under Cooperative Agreement Number W911NF-20-2-0267, NSF CAREER Award CCF-2144927, and the UCR OASIS Fellowship. The views in this document are those of the authors and should not be interpreted as representing the official policies of the ARL and USD(R&E)/RT&L or the U.S. Government.

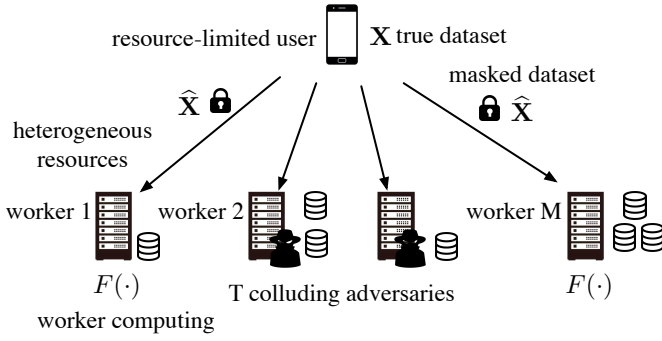


Fig. 1. Secure computation offloading.

II. PROBLEM FORMULATION

We consider a wireless computation offloading scenario illustrated in Fig.1, where a resource-limited user offloads computations to M edge servers (workers). The user has a local dataset $\mathbf{X} \in \mathbb{R}^{l \times d_1}$, where l denotes the total number of data samples and d_1 denotes the number of features for each sample, respectively. The user wants to evaluate a degree- r multivariate polynomial function $F(\mathbf{X}_i) \in \mathbb{R}^{d_2}$ for $i \in [l]$ by offloading the computations to the workers, where $\mathbf{X}_i$ denotes the i -th data sample, i.e., row i of $\mathbf{X}$.

We consider a two-phase offline-online communication scenario, where data-agnostic operations such as randomness generation can be carried out in a delay-tolerant offline phase when the network load is low [12]. During the offline phase, we assume the existence of secure point-to-point links between the user and each worker, where no eavesdropper can learn the transmitted messages. Data-dependent operations are carried out in a delay-sensitive online phase. During the online phase, adversaries can eavesdrop on any communication link between the user and workers.

Threat model. We consider a passive (honest-but-curious) adversary model, where adversaries do not modify the protocol but may attempt to learn additional information about the dataset and computation results from the messages they observe. We assume that any set of up to $T < M$ workers may be adversarial, who may collude with each other.

Main problem. Our main goal is to enable secure computation offloading while minimizing the end-to-end task completion time under resource heterogeneities. As the data-agnostic offline operations can be carried out in advance when network load is low, our focus is on minimizing the online (data-dependent) computation time. To do so, the user initially selects a subset of workers $\mathcal{S} \subseteq [M]$ to offload the computations. Then, secure coded computation offloading operates in three stages:

- 1) *Dataset offloading:* Initially, the user sends a *masked dataset* $\hat{\mathbf{X}}$ to the selected workers. The goal of dataset masking is to prevent the workers from learning the true value of the dataset.
- 2) *Worker computing:* After receiving the masked dataset $\hat{\mathbf{X}}$, each selected worker locally computes the function $F(\cdot)$ (without observing the true dataset $\mathbf{X}$), and sends their results back to the user.

- 3) *Decoding final results:* After receiving the local computations from the workers, the user recovers the desired computation $F(\mathbf{X}_i)$ for each data sample $i \in [l]$.

Our goal is to find the optimal worker selection and computation allocation strategy to minimize the end-to-end online task completion time,

$$\min \left(t^O + \max_{i \in \mathcal{S}} t_i^W + t^D \right) \quad (1)$$

where t^O denotes the time cost of the dataset offloading stage, which includes generating the masked dataset $\hat{\mathbf{X}}$ and sending it to the selected workers, t_i^W denotes the time cost for the local computation stage at worker i , which includes computing the function $F(\cdot)$ and sending the result back to the user, and t^D denotes the time cost of decoding the computation results at the user. In doing so, the information-theoretic privacy of the dataset and computation results should be preserved against any set of T adversarial workers, which can be denoted by the following mutual information condition,

$$I(\mathbf{X}, \{F(\mathbf{X}_i)\}_{i \in [l]}; \mathcal{A}|F) = 0 \quad (2)$$

where $\mathcal{A}$ denotes the collection of the messages observed or generated by the adversaries, which includes all online communication between the user and workers (honest or adversarial), offline (point-to-point) messages received by the adversaries from the user, and the messages generated by the adversaries in online or offline phases.

To satisfy the information-theoretic privacy guarantees from (2), secure coded computing mechanisms mainly operate in a finite field, where the dataset $\mathbf{X}$ is converted to the finite field using a simple unbiased stochastic quantization mechanism [7], [11], [12]. In the following, we also assume that all operations are carried out in a finite field $\mathbb{F}_p$ of integers modulo a large prime p , and let $\mathbf{X}$ represent the finite field representation of the dataset. Due to space limitations, we refer to [12] for the details of this conversion. We next introduce our secure computation offloading framework to solve (1) under the information-theoretic privacy guarantees from (2).

III. SECURE COMPUTATION OFFLOADING WITH DOUBLE LAGRANGE CODING

In this section, we introduce our secure computation offloading framework. Our framework is motivated by the recent secure computing mechanism DLC [11], [12], which achieves an order or magnitude improvement on the communication and computation complexity for complex machine learning tasks compared to conventional secure coded computing baselines. On the other hand, the original DLC framework is designed for a decentralized peer-to-peer communication setup with homogeneous resources, as opposed to our computation offloading task with heterogeneous resources. Our secure offloading framework consists of the following phases.

A. Offline Phase

The offline phase consists of randomness generation and takes place in advance when network load is low. During this

phase the user can communicate with each worker through a secure point-to-point communication link.

Initially, the user and workers agree on two sets of public parameters, $\{\alpha_j\}_{j \in [M]}$ and $\{\{\beta_i\}_{i \in [r(K+T-1)+1]}\}_{K \in [\bar{K}]}$ from $\mathbb{F}_p$, where $\bar{K} \triangleq \lfloor \frac{M-1}{r} - T + 1 \rfloor$ and $\alpha_j \neq \beta_i$ for all j, i .

For each $K \in [\bar{K}]$, the user generates two sets of random masks¹ $\mathbf{R}_1, \dots, \mathbf{R}_K \in \mathbb{F}_p^{\frac{1}{K} \times d_1}$ and $\mathbf{U}_1, \dots, \mathbf{U}_K \in \mathbb{F}_p^{\frac{1}{K} \times d_2}$, where each element is generated independently and uniformly at random from $\mathbb{F}_p$. The user then encodes the random masks using two Lagrange interpolation polynomials,

$$\begin{aligned} \tilde{\mathbf{R}}_j &= \sum_{k=1}^K \mathbf{R}_k \prod_{i=1, i \neq k}^{K+T} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} \\ &+ \sum_{k=K+1}^{K+T} \mathbf{V}_k \prod_{i=1, i \neq k}^{K+T} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} \quad \forall j \in [M], \end{aligned} \quad (3)$$

and,

$$\begin{aligned} \tilde{\mathbf{U}}_j &= \sum_{k=1}^K \mathbf{U}_k \prod_{i=1, i \neq k}^{r(K+T-1)+1} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} \\ &+ \sum_{k=K+1}^{r(K+T-1)+1} \mathbf{M}_k \prod_{i=1, i \neq k}^{r(K+T-1)+1} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} \quad \forall j \in [M], \end{aligned} \quad (4)$$

where $\mathbf{V}_{K+1}, \dots, \mathbf{V}_{K+T} \in \mathbb{F}_p^{\frac{1}{K} \times d_1}$ and $\mathbf{M}_{K+1}, \dots, \mathbf{M}_{r(K+T-1)+1} \in \mathbb{F}_p^{\frac{1}{K} \times d_2}$ are random matrices with each element generated independently and uniformly at random in $\mathbb{F}_p$. Finally, user sends the encoded masks $\tilde{\mathbf{R}}_j$ and $\tilde{\mathbf{U}}_j$ to user $j \in [M]$ through the secure point-to-point communication link.

Key intuition: The two encoded masks will later be used by the user in the online phase to hide the true values of the dataset and computation results from the adversarial workers. The first mask $\mathbf{R}_1, \dots, \mathbf{R}_K$ will hide the true value of the dataset, whereas the second mask $\mathbf{U}_1, \dots, \mathbf{U}_K$ will hide the true value of the computation results, as will be shown in the online phase. In doing so, encoding the masks with the random matrices $\mathbf{V}_{K+1}, \dots, \mathbf{V}_{K+T}$ and $\mathbf{M}_{K+1}, \dots, \mathbf{M}_{r(K+T-1)+1}$ will hide the true values of the masks from the workers.

B. Online Phase

In the online phase, the user first selects a set of workers $\mathcal{S} \subseteq [M]$ where $|\mathcal{S}| = r(K+T-1) + 1$, and a hyperparameter $K \in [\bar{K}]$. The worker and hyperparameter selection mechanism will be detailed in the following section when we introduce our resource-aware secure offloading mechanism. Parameter K allows tuning the local computing workload at the workers and provides a trade-off between the worker computing load and the decoding load at the user; a larger K will reduce the local computation load at the workers, at the cost of increased decoding complexity at the user. In the following, K will be referred to as the *load distribution degree*.

During the online phase, adversaries can observe any communication link. The online phase consists of three stages.

¹Without loss of generality, we assume $l \mid K$ for $K \in [\bar{K}]$. If l is not divisible by K , one can use the smallest l such that $l \mid K$ for $K \in \bar{K}$.

1) *Dataset Offloading:* To hide the true dataset during offloading, the user multi-casts a masked dataset,

$$\hat{\mathbf{X}} = \mathbf{X} - \mathbf{R} \quad (5)$$

to the selected workers $\mathcal{S}$, using the random mask $\mathbf{R} \triangleq [\mathbf{R}_1^T \dots \mathbf{R}_K^T]$ generated in the offline phase. The random mask $\mathbf{R}$ hides the true content of $\mathbf{X}$ from the workers

2) *Worker Computing:* After dataset offloading, workers need to compute the desired polynomial function $F(\cdot)$. To enable higher order polynomial computations, worker $j \in \mathcal{S}$ partitions the masked dataset into K equal-sized shards $\hat{\mathbf{X}} = (\hat{\mathbf{X}}_1, \dots, \hat{\mathbf{X}}_K)$, and encodes them into a polynomial by using the encoded mask $\tilde{\mathbf{R}}_j$ generated in (3) from the offline phase,

$$\tilde{\mathbf{X}}_j = \sum_{k=1}^K \hat{\mathbf{X}}_k \prod_{i=1, i \neq k}^{K+T} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} + \tilde{\mathbf{R}}_j \quad (6)$$

$$\begin{aligned} &= \sum_{k=1}^K \mathbf{X}_k \prod_{i=1, i \neq k}^{K+T} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i} \\ &+ \sum_{k=K+1}^{K+T} \mathbf{V}_k \prod_{i=1, i \neq k}^{K+T} \frac{\alpha_j - \beta_i}{\beta_k - \beta_i}, \end{aligned} \quad (7)$$

where the random masks $\{\mathbf{R}_k\}_{k \in [K]}$ cancel out in (6), and the dataset, partitioned into K shards $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_K)$, is embedded into a Lagrange polynomial of degree $K+T-1$ as shown in (7). The random matrices $\mathbf{V}_{K+1}, \dots, \mathbf{V}_{K+T}$ prevent the workers from learning the true dataset $\mathbf{X}$ during encoding.

Using the encoded dataset $\tilde{\mathbf{X}}_j$, worker j locally computes the desired function $F(\tilde{\mathbf{X}}_j)$. Before sending the result back to the user, the worker uses the second encoded mask $\tilde{\mathbf{U}}_j$ from (4) to mask the local computation $F(\tilde{\mathbf{X}}_j)$, and sends the masked computation $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j$ to the user.

Note that online communication is carried out over insecure links, which can be observed by adversaries. To that end, the second mask $\tilde{\mathbf{U}}_j$ ensures that an adversary cannot learn the true computation results $F(\mathbf{X}_1), \dots, F(\mathbf{X}_K)$ even if they eavesdrop on every online communication link, and learn $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j$ for all $j \in \mathcal{S}$.

3) *Decoding Final Results:* After receiving the encoded computation results $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j$ from workers $j \in \mathcal{S}$, the user decodes the true computation results $F(\mathbf{X}_1), \dots, F(\mathbf{X}_K)$. From (6), $\tilde{\mathbf{X}}_j$ can be viewed as an evaluation point of a degree- $(K+T-1)$ polynomial $\phi(\alpha)$ where $\tilde{\mathbf{X}}_j = \phi(\alpha_j)$. Since F is a degree- r polynomial, $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j$ can also be viewed as an evaluation point of a degree- $r(K+T-1)$ polynomial $\psi(\alpha)$ where $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j = \psi(\alpha_j)$. In order to decode the true computation results $F(\mathbf{X}_1), \dots, F(\mathbf{X}_K)$, the user can then interpolate $\psi(\alpha)$ by collecting $F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j$ from at least $|\mathcal{S}| \geq r(K+T-1) + 1$ workers, and recover $\psi(\beta_k) = F(\mathbf{X}_k) - \mathbf{U}_k$ as,

$$\psi(\beta_k) = \sum_{j \in \mathcal{S}} \left(F(\tilde{\mathbf{X}}_j) - \tilde{\mathbf{U}}_j \right) \prod_{j' \in \mathcal{S}, j' \neq j} \frac{\beta_k - \alpha_{j'}}{\alpha_j - \alpha_{j'}} \quad (8)$$

$$= F(\mathbf{X}_k) - \mathbf{U}_k \quad (9)$$

after which the true result $F(\mathbf{X}_k)$ can be decoded as $F(\mathbf{X}_k) = \psi(\beta_k) + \mathbf{U}_k$ for $k \in [K]$, since the masks $\mathbf{U}_1, \dots, \mathbf{U}_K$ are known by the user.

Note that the interpolation operation in (9) requires the computations to be collected from at least $|\mathcal{S}| \geq r(K+T-1)+1$ workers for correct decoding. Since $|\mathcal{S}| \leq M$, this also provides an upper bound $K \leq \lceil \frac{M-1}{r} - T + 1 \rceil = \bar{K}$.

IV. HIERARCHICAL NEURAL BANDITS FOR RESOURCE-AWARE SECURE OFFLOADING

Our goal is to minimize the end-to-end latency while maintaining information-theoretic privacy for the user dataset $\mathbf{X}$ and computation results $F(\mathbf{X}) = (F(\mathbf{X}_1), \dots, F(\mathbf{X}_K))$ as shown in (2). We observe that increasing the load distribution degree K reduces the worker computation overhead, as each worker only needs to process $1/K^{th}$ of the dataset $\mathbf{X}$. However, increasing K also increases the decoding complexity at the user. Since the user is resource-limited, selecting an optimal K is crucial to balancing the computation trade-off between the user and workers.

To model the heterogeneous computation capabilities, we let f^U denote the CPU frequency of the user, and f_j^W denote the CPU frequency of worker $j \in [M]$, where $f^U \ll f_j^W$. Since different workers may have varying computational resources, it is necessary to choose the optimal set of workers for efficient computation offloading. We assume that the CPU frequencies of the workers are unknown to the user.

The user aims at determining the optimal workload allocation K and the set of workers $\mathcal{S}$ of size $|\mathcal{S}| \geq r(K+T-1)+1$ to participate in offloading to minimize the end-to-end latency, which can be formulated as,

$$\begin{aligned} \min_{K, a_1, \dots, a_M} \quad & \left(t^O + \max_{j \in [M]} \{a_j \cdot t_j^W\} + t^D \right) \\ \text{subject to} \quad & K \in \{1, \dots, \bar{K}\} \\ & \sum_{j \in [M]} a_j \geq r \cdot (K + T - 1) + 1 \\ & a_j \in \{0, 1\}, \forall j \in [M] \end{aligned} \quad (10)$$

where the binary variable $a_j \in \{0, 1\}$ indicates whether worker $j \in [M]$ is selected ($a_j = 1$ if selected) by the user.

Since the computation latency of (5) is independent from K and $a_1, \dots, a_M$, for the dataset offloading latency we consider the communication latency for multi-casting the masked dataset, and let $t^O = ld_1 b / R^O$, where $b = \lceil \log_2 p \rceil$ is the number of bits used to represent each element in $\mathbb{F}_p$,

$$R^O = B \log \left(1 + \frac{H_{\min} P^U}{\sigma^2} \right) \quad (11)$$

is the rate for multi-casting to the set of workers $\mathcal{S}$, B is the total bandwidth, $H_{\min} = \min_{j \in [M]: a_j=1} H_j$ where H_j is the channel gain between the user and worker $j \in [M]$, P^U is the transmit power of the user, and σ^2 is the noise power. We assume that perfect channel state information (CSI) is available at the user [16] [17].

The processing latency t_j^W for worker j consists of local computation and communicating the results to the user. Worker-to-user communication uses OFDMA (Orthogonal Frequency Division Multiple Access), and the total bandwidth is divided equally across the selected workers $\mathcal{S}$. The communication latency for worker j is given by $ld_2 b / K R_j^W$, where

$$R_j^W = \frac{B}{|\mathcal{S}|} \log \left(1 + \frac{H_j P_j^W}{\sigma^2} \right) \quad (12)$$

denotes the rate from worker j to the user, P_j^W is the transmit power of worker j . On the other hand, due to the lack of knowledge about the computation resources $\{f_j^W\}_{j \in [M]}$ of the workers, the local computation latency of each worker is unknown to the user, which prevents conventional approaches such as Branch and Bound to be applied to solve the integer program in (10). To address this challenge, we next propose a hierarchical neural bandit approach for resource-aware secure offloading under heterogeneous resources.

Hierarchical Neural MAB for Secure Offloading. We first decompose (10) into two sub-problems: 1) Workload allocation (K), and 2) Worker selection ($\mathcal{S}$), each of which can be reformulated as a neural bandit problem. The goal of the user is to learn the optimal workload allocation and worker selection strategy within N rounds.

1) *Workload Allocation:* As described in Section III, worker load allocation is controlled by parameter K . To that end, we first transform the workload allocation task into a $\bar{K}$ -armed neural MAB. We let each arm correspond to a potential load distribution degree $K \in \{1, \dots, \bar{K}\}$, where the user acts as the agent for arm selection. At round n , the user observes a context vector for arm K , defined as:

$$\mathbf{c}_{n,K}^1 = [H_{n,1}, \dots, H_{n,M}, \boldsymbol{\tau}_K^1], \quad (13)$$

where $H_{n,j}$ represents the channel gain between the user and worker j at round n , and $\boldsymbol{\tau}_K^1$ is a one-hot indicator vector for K . The user then computes the upper confidence bound (UCB) score for each arm K [13],

$$U_{n,K}^1 = h_1(\mathbf{c}_{n,K}^1; \boldsymbol{\theta}_{n-1}^1) + \gamma \sqrt{\nabla h_1^\top (\mathbf{Z}_{n-1}^1)^{-1} \nabla h_1} \quad (14)$$

where h_1 is the output (estimated reward) of a neural network parameterized by $\boldsymbol{\theta}^1$, and ∇h_1 represents the gradient. $\mathbf{Z}_{n-1}^1$ is a Gram matrix that aggregates the context information for the selected arms up to round $n-1$, and γ is a positive scaling factor. The user then selects the arm with the highest score,

$$K_n = \arg \max_{K \in [\bar{K}]} U_{n,K}^1 \quad (15)$$

2) *Worker Selection:* After selecting the load distribution degree K_n , the user then performs worker selection. We transform the worker selection problem into an M -armed combinatorial MAB, where each arm represents a worker, and the user is the agent responsible for choosing the workers. At round n , the user observes the context of arm j defined as,

$$\mathbf{c}_{n,j}^2 = [H_{n,j}, \boldsymbol{\tau}_{K_n}^1, \boldsymbol{\tau}_j^2] \quad (16)$$

where τ_j^2 is a one-hot vector indicating the current arm j , and $\tau_{K_n}^1$ is a one-hot vector indicating the load distribution degree K_n selected by the first MAB. Then, the user computes the UCB score of each arm,

$$U_{n,j}^2 = h_2(\mathbf{c}_{n,j}^2; \boldsymbol{\theta}_{n-1}^2) + \gamma \sqrt{\nabla h_2^\top (\mathbf{Z}_{n-1}^2)^{-1} \nabla h_2} \quad (17)$$

where h_2 denotes the output of a neural network parameterized by $\boldsymbol{\theta}^2$, and ∇h_2 represents the gradient. $\mathbf{Z}_{n-1}^2$ is the Gram matrix that aggregates the context information for the selected arms up to round $n-1$. The user then selects the set of arms (super-arm) $\mathcal{S}_n \subseteq [M]$ corresponding to the top $|\mathcal{S}_n| = r(K_n + T - 1) + 1$ scores from (17).

Neural Bandit Training. The neural bandits are trained to estimate the reward of each arm given the context information. We consider an offline training and online testing strategy. For workload allocation, the reward at round n is defined as,

$$r_n = \exp(-(t_n^O + \max_{j \in \mathcal{S}_n} \{t_{n,j}^W\} + t_n^D)). \quad (18)$$

For worker selection, the reward for arm j at round n is²,

$$v_{n,j} = \exp(-(t_n^O + a_{n,j} \cdot t_{n,j}^W + t_n^D)) \quad (19)$$

where we define the reward of the super-arm $\mathcal{S}_n$ as,

$$r(\mathcal{S}_n, \mathbf{v}_n) = \min_{j \in \mathcal{S}_n} v_{n,j} \quad (20)$$

After calculating the rewards from (18) and (19), the user trains the neural bandits using gradient descent,

$$\boldsymbol{\theta}^j \leftarrow \boldsymbol{\theta}^j - \eta \nabla L(\boldsymbol{\theta}^j) \text{ for } j \in \{1, 2\} \quad (21)$$

where η is the learning rate, and $L(\cdot)$ is a quadratic loss function defined as,

$$L(\boldsymbol{\theta}^1) = \frac{1}{2} \sum_{n=1}^N (h_1(\mathbf{c}_{n,K_n}^1; \boldsymbol{\theta}^1) - r_n)^2 + \Omega(\boldsymbol{\theta}^1) \quad (22)$$

for workload allocation, and

$$L(\boldsymbol{\theta}^2) = \frac{1}{2} \sum_{n=1}^N \sum_{j \in \mathcal{S}_n} (h_2(\mathbf{c}_{n,j}^2; \boldsymbol{\theta}^2) - v_{n,j})^2 + \Omega(\boldsymbol{\theta}^2) \quad (23)$$

for worker selection, respectively, where $\Omega(\cdot)$ is an L2-regularization term. As we demonstrate in our experiments, this hierarchical structure allows for a more efficient search process and significantly outperforms using single MAB.

V. THEORETICAL ANALYSIS

Our framework provides information-theoretic privacy guarantees for both the dataset and computation results.

Theorem 1 (Information-theoretic privacy). *To compute a polynomial F of degree r in a network of M workers, our framework guarantees information-theoretic privacy for both the dataset $\mathbf{X}$ and computation results $F(\mathbf{X}_i)$ for $i \in [l]$, against any set of up to T adversarial workers,*

$$I(\mathbf{X}, \{F(\mathbf{X}_i)\}_{i \in [l]}; \mathcal{A}|F) = 0 \quad (24)$$

²We consider a semi-bandit feedback setting, where the user observes the reward for each arm $j \in \mathcal{S}_n$.

as long as $M \geq r(K + T - 1) + 1$, where $\mathcal{A}$ denotes the collection of all messages received or generated by the adversaries, and F represents the computation task.

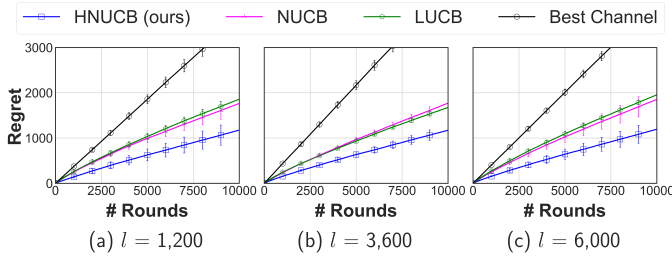
Proof (Sketch). Due to space limitations we provide a proof sketch. Information-theoretic privacy of the dataset $\mathbf{X}$ follows from the privacy guarantees of Double Lagrange Coding, which follows from [12]. The privacy guarantees of the final results $F(\mathbf{X}_i)$ for $i \in [l]$ are due to the additive random masks $\mathbf{U}_1, \dots, \mathbf{U}_K$ which are known only to the user, that mask the true computations $F(\mathbf{X}_1), \dots, F(\mathbf{X}_K)$ for the K shards $\mathbf{X}_1, \dots, \mathbf{X}_K$ of the dataset $\mathbf{X}$. Hence, the final results $F(\mathbf{X}_1), \dots, F(\mathbf{X}_K)$ can be decoded only by the user. $\square$

VI. EXPERIMENTS

Setup. We consider an inference task to classify images from CIFAR-10 [18], using a (public) pre-trained polynomial neural network (PiNet) from [15], with a polynomial degree $r = 2$. We consider a network with 11 workers randomly distributed within a $500\text{m} \times 500\text{m}$ area. The channel gain H_j between the user and worker j accounts for both large-scale and small-scale fading. The path loss is defined as $128.1 + 37.6 \log_{10}(\text{dist}_j)$, where dist_j represents the distance (in kilometers) between the user and worker j . The noise power is -96 dBm , and the bandwidth is 2 MHz . The transmit power for both the user and workers is 23 dBm . The CPU frequency of the user is 10 MHz , while the CPU frequencies of the workers follows a Beta distribution between 100 to 1000 MHz . Specifically, the CPU frequency of worker j is given by $f_j^W = 100 + \text{Beta}(\mu_1, \mu_2) \cdot (1000 - 100)$, where $\text{Beta}(\mu_1, \mu_2)$ represents a random variable drawn from a Beta distribution with shape parameters $\mu_1 = 0.5$ and $\mu_2 = 0.5$, scaled to the range $[100, 1000]\text{ MHz}$. The number of adversarial workers is $T = 1$, whereas the finite field size is $p = 2^{26} - 5$.

Baselines. We consider the following baselines for performance evaluation.

- **Hierarchical Neural UCB (HNUCB, ours):** We employ two 2-layer fully connected neural networks with a width of 32 for our hierarchical neural bandit, one for worker selection ($M = 11$) and one for computation load optimization ($\bar{K} = 5$), respectively.
- **Neural UCB (NUCB) [13]:** The user implements a single neural contextual bandit with UCB exploration for joint load allocation and worker selection. Each arm corresponds to a joint load allocation (K) and worker selection ($\mathcal{S}$) action, resulting in a total number of $1,013$ arms. The context vector for arm (K, j) at round n is defined as $[H_{n,j}, \tau_K^1, \tau_j^2]$ where $K \in [\bar{K}]$, $j \in [M]$. For the neural network, we utilize the same architecture as HNUCB, but with a slightly larger width of 64 to accommodate the larger dimension of the context vector.
- **Linear Contextual Bandit (LUCB) [19], [20]:** The user implements conventional linear contextual bandits (as opposed to neural bandits) using the same hierarchical structure and contexts from HNUCB. This compares the performance between linear vs. neural bandits.

Fig. 2. Training performance for varying user dataset size (l).

- **Best Channel** [10]: User sets the load distribution level to $K = 1$ and selects the set of $|\mathcal{S}| = r(K + T - 1) + 1 = 3$ workers with the best channels. Setting $K = 1$ corresponds to well-known Shamir's Secret Sharing [10].

Evaluation Metric. During training, performance is evaluated using the cumulative regret, where the optimal reward in each round is evaluated through exhaustive search, where we search for the optimal solution that provides the minimum system latency across all computation load allocations K and worker selections $\mathcal{S}$. Note that exhaustive search is not feasible in practice as the worker computation resources are not known to the user apriori, hence is provided solely for comparison purposes. In the test phase, we measure the average system latency. The number of training and test rounds for the bandits are set to 10,000 and 5,000, respectively. The results are averaged based on three independent runs of the experiments.

Performance. Fig. 2 shows the cumulative regret of all baselines during training, as the number of images processed (l) increases. We observe that *HNUCB* consistently outperforms the other baselines. This can be attributed to *HNUCB*'s ability to efficiently capture complex relationships between the context and actions through the hierarchical neural architecture. A comparison of our hierarchical framework (*HNUCB*) with the conventional neural bandit (*NUCB*) supports this intuition, as *NUCB* has to explore a significantly larger number of arms due to its complex action space. In contrast, the efficient exploration strategy in *HNUCB* helps balance exploration and exploitation more effectively, achieving lower regret. We observe that *Best Channel* performs the worst among all schemes, highlighting the necessity of learning adaptive computation load at the workers, and that selecting the workers with the best channel conditions is not sufficient.

Table I illustrates the average system latency for all baselines during test-time, for varying dataset size l . For comparison, we also include the latency for exhaustive search (optimal), and local computation at the user (termed *Local*), where the user executes the computation locally, without offloading to the workers. Note that in this case, computations can be performed on the clear data, without the overhead for secure computation. We observe that *HNUCB* again outperforms the other baselines and achieves near-optimal system latency.

VII. CONCLUSION

We propose an adaptive secure offloading framework for resource-aware computation offloading. Our framework ensures information-theoretic privacy for both the user data

TABLE I
TEST PERFORMANCE ACROSS THE BASELINES.

Schemes	Average system latency (seconds)		
	$l = 1, 200$	$l = 3, 600$	$l = 6, 000$
<i>HNUCB</i> (ours)	175.9 ± 11.0	536.2 ± 35.0	880.2 ± 55.4
<i>NUCB</i>	198.5 ± 9.6	699.7 ± 56.6	1045.0 ± 114.9
<i>LUCB</i>	205.3 ± 13.3	648.5 ± 41.5	1068.2 ± 56.3
<i>Best Channel</i>	504.7 ± 45.5	1526.7 ± 136.3	2542.2 ± 226.6
<i>Local</i>	2242.6 ± 0.0	6914.3 ± 0.0	11453.6 ± 0.0
<i>Exhaustive search</i>	125.4 ± 13.0	389.9 ± 39.1	649.0 ± 65.3

and computation results, while our experiments demonstrate significant improvement across conventional baselines.

REFERENCES

- [1] F. Shirin, P. Ramezani, S. Iranmanesh, S. Murali, D. Chulerttiyawong, X. Wan, A. Jamalipour, and R. Raad, "A survey on mobility of edge computing networks in IoT: State-of-the-art, architectures, and challenges," *IEEE Comm. Surveys & Tutorials*, vol. 24, no. 4, 2022.
- [2] D. Wen, M. Bennis, and K. Huang, "Joint parameter-and-bandwidth allocation for improving the efficiency of partitioned edge learning," *IEEE Trans. on Wireless Comm.*, vol. 19, no. 12, pp. 8272–8286, 2020.
- [3] C. Yang, R. Pedarsani, and A. S. Avestimehr, "Edge computing in the dark: Leveraging contextual-combinatorial bandit and coded computing," *IEEE/ACM Trans. on Netw.*, vol. 29, no. 3, pp. 1022–1031, 2021.
- [4] X. He, T. Li, R. Jin, and H. Dai, "Delay-optimal coded offloading for distributed edge computing in fading environments," *IEEE Trans. on Wireless Comm.*, vol. 21, no. 12, pp. 10796–10808, 2022.
- [5] E. Michailidis, M. Volakaki, N. Miridakis, and D. Vouyioukas, "Optimization of secure computation efficiency in UAV-enabled RIS-assisted MEC-IoT networks with aerial and ground eavesdroppers," *IEEE Trans. on Comm.*, vol. 72, no. 7, pp. 3994–4009, 2024.
- [6] X. Lu, H. Sami, and B. Güler, "Privacy-preserving collaborative learning with linear communication complexity," *IEEE Trans. Inf. Theory*, 2023.
- [7] J. So, B. Güler, and S. Avestimehr, "Codedprivateml: A fast and privacy-preserving framework for distributed machine learning," *IEEE J. Sel. Areas Inf. Theory*, vol. 2, no. 1, pp. 441–451, 2021.
- [8] Q. Yu, S. Li, N. Raviv, S. Kalan, M. Soltanolkotabi, and S. Avestimehr, "Lagrange coded computing: Optimal design for resiliency, security, and privacy," in *Int. Conf. on Artificial Int. and Statistics*, 2019.
- [9] R. Schlegel, S. Kumar, E. Rosnes, and A. Graell, "Privacy-preserving coded mobile edge computing for low-latency distributed inference," *IEEE J. Sel. Areas Comm.*, vol. 40, no. 3, pp. 788–799, 2022.
- [10] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, 1979.
- [11] X. Lu and B. Güler, "Breaking the quadratic communication overhead of secure multi-party neural network training," in *2023 IEEE International Symposium on Information Theory (ISIT)*, 2023, pp. 921–926.
- [12] X. Lu, U. Y. Basaran, and B. Güler, "Scalable multi-round multi-party privacy-preserving neural network training," *IEEE Transactions on Information Theory*, vol. 70, no. 11, pp. 8204–8236, 2024.
- [13] D. Zhou, L. Li, and Q. Gu, "Neural contextual bandits with ucb-based exploration," in *Int. Conf. on Machine Learning, ICML*, 2020.
- [14] T. Hwang, K. Chai, and M. Oh, "Combinatorial neural bandits," in *Int. Conf. on Machine Learning, ICML*, 2023, pp. 14203–14236.
- [15] G. Chrysos, S. Moschoglou, G. Bouritsas, J. Deng, Y. Panagakis, and S. Zafeiriou, "Deep polynomial neural networks," *IEEE Trans. on Pattern Analysis and Machine Int.*, vol. 44, no. 8, pp. 4021–4034, 2022.
- [16] C. Deng, X. Fang, and X. Wang, "UAV-enabled mobile-edge computing for AI applications: Joint model decision, resource allocation, and trajectory optimization," *IEEE Internet Things J.*, 2023.
- [17] Y. Liu, X. Fang, M. Xiao, F. Song, Y. Cui, Q. Xue, and C. Tang, "Latency optimization for multi-UAV-assisted task offloading in air-ground integrated millimeter-wave networks," *IEEE Trans. on Wireless Comm.*, pp. 1–1, 2024.
- [18] A. Krizhevsky, "Learning multiple layers of features from tiny images," *University of Toronto*, 05 2012.
- [19] L. Li, W. Chu, J. Langford, and R. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *WWW*, 2010.
- [20] L. Qin, S. Chen, and X. Zhu, "Contextual combinatorial bandit and its application on diversified online recommendation," in *ICDM*, 2014.